

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Specifica Procedure & Abstract Data Type

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

TdE 28/08/2023

- Dato il seguente codice, rispondere ai quesiti successivi fornendo specifiche JML

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }
```

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }
```

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }
```

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}
```

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }
```

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }
```

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

TdE: 28-08-2023 Esercizio 1 – quesito a

Si specifichi in JML il metodo getSitesByType()

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires**

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null

Dal momento che non è contemplato il caso «null» per nessuno dei due input, la precondizione che consideriamo è che **ogni** input sia diverso da null

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null

- **@ signals**

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null

Partiamo dai comportamenti «semplici». Sappiamo che, se non esiste un sito, tra quelli contenuti, che ha il tipo specificato in ingresso, allora restituisco un'eccezione.

- **@ signals** (NotFoundException e)
  !(\exists Site s; getSites().contains(s); s.getType().equals(type))

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null
- **@ ensures**

- **@ signals** (NotFoundException e)
  !(\exists Site s; getSites().contains(s); s.getType().equals(type))

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null
- **@ ensures** (\exists Site s; getSites().contains(s); s.getType().equals(type)) && \result!=null &&

La prima cosa da fare, se ho una signal, è fare in modo che sia in **mutua esclusione** con la ensures, quindi il primo statement della ensure, è opposto a quello della Signal. Inoltre, dato che esiste almeno un Site s, sicuramente avremo result diverso da null

- **@ signals** (NotFoundException e) !(\exists Site s; getSites().contains(s); s.getType().equals(type))

Mutua Esclusione

Requires: indicano ciò che deve essere vero prima della chiamata affinchè il metodo possa essere usato correttamente

Ensures: Descrivono **ciò che deve essere vero dopo la chiamata**, **solo se il metodo termina normalmente** (cioè senza lanciare eccezioni).

Signals: Descrivono **ciò che deve essere vero dopo la chiamata**, **solo quando il metodo termina lanciando un'eccezione** del tipo indicato.

Mutua Esclusione

Requires: indicano ciò che deve essere vero prima della chiamata affinchè il metodo possa essere usato correttamente

Ensures: Descrivono **ciò che deve essere vero dopo la chiamata**, **solo se il metodo termina normalmente** (cioè senza lanciare eccezioni).

Signals: Descrivono **ciò che deve essere vero dopo la chiamata**, **solo quando il metodo termina lanciando un'eccezione** del tipo indicato.

Requires e **Signal** ***possono essere*** in mutua esclusione in tutti quei casi in cui c'è un errore da parte dell'utente E si lancia eccezione

Mutua Esclusione

Requires: indicano ciò che deve essere vero prima della chiamata affinchè il metodo possa essere usato correttamente

Ensures: Descrivono **ciò che deve essere vero dopo la chiamata**, **solo se il metodo termina normalmente** (cioè senza lanciare eccezioni).

Signals: Descrivono **ciò che deve essere vero dopo la chiamata**, **solo quando il metodo termina lanciando un'eccezione** del tipo indicato.

Requires e **Signal** ***possono essere*** in mutua esclusione in tutti quei casi in cui c'è un errore da parte dell'utente E si lancia eccezione

Ensures e **Signal** ***devono essere*** in mutua esclusione in quanto le ensures sono post-condizioni in caso di esecuzione terminata normalmente, mentre le signal sono postcondizioni in caso di esecuzione terminata con eccezione

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null
- **@ ensures** (\exists Site s; getSites().contains(s); s.getType().equals(type)) && \result!=null &&

- **@ signals** (NotFoundException e) !(\exists Site s; getSites().contains(s); s.getType().equals(type))

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null
- **@ ensures** (\exists Site s; getSites().contains(s); s.getType().equals(type)) && \result!=null &&

(\forall Site s; ; \result.contains(s) <==> getSites().contains(s) && s.getType().equals(type)) &&

Dal momento che « Ritorna tutti i punti di interesse del tipo passato per parametro», devo controllare che **ogni sito** sia contenuto nel risultato **se e solo se** è un sito che esiste nel Set<Site> ed il tipo è uguale a quello passato come parametro

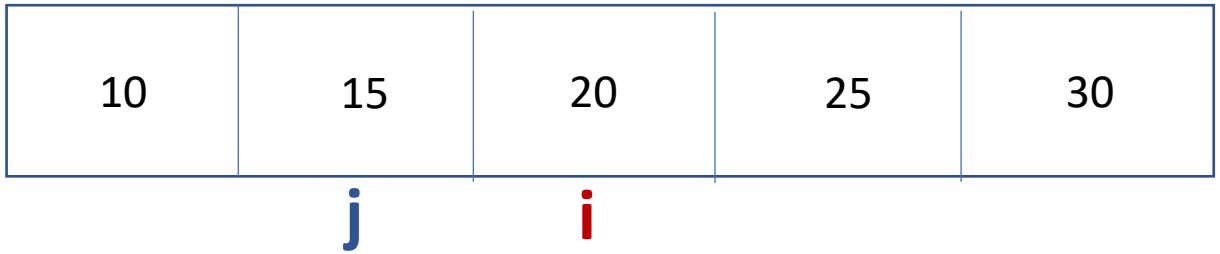
- **@ signals** (NotFoundException e) !(\exists Site s; getSites().contains(s); s.getType().equals(type))

TdE: 28-08-2023 Esercizio 1 – soluzione a

La traccia dice:
Ordinati per distanza CRESCENTE rispetto alla location indicata.

Ciò signica che, per ogni elemento, devo controllare sia PIU GRANDE, di tutti i suoi precedenti (o più piccolo, di tutti i successivi).

Mi servono quindi due indici, i e j. Se consideriamo i verso la fine della lista, quindi come il più grande di tutti i precedenti, mi serve che j sia sempre **MINORE di i, cosi da puntare solo ai precedenti.**



TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null
- **@ ensures** (\exists Site s; getSites().contains(s); s.getType().equals(type)) && \result!=null &&

(\forall Site s; ; \result.contains(s) <==> getSites().contains(s) && s.getType().equals(type)) &&

- ***@ signals*** (NotFoundException e) !(\exists Site s; getSites().contains(s); s.getType().equals(type))

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null
- **@ ensures** (\exists Site s; getSites().contains(s); s.getType().equals(type)) && \result!=null &&

(\forall Site s; ; \result.contains(s) <==> getSites().contains(s) && s.getType().equals(type)) &&

(\forall int i; i>=0 && i<\result.size();

(\forall int j; j>=0 && j<i; !\result.get(j).equals(\result.get(i)) &&

In questo modo, sono certo che l'elemento a cui si riferisce j sarà **diverso** dall'elemento a cui si riferisce i

- ***@ signals*** (NotFoundException e) !(\exists Site s; getSites().contains(s); s.getType().equals(type))

TdE: 28-08-2023 Esercizio 1 – soluzione a

La traccia dice:
Ordinati per distanza CRESCENTE rispetto alla location indicata.

Ciò signica che, per ogni elemento, devo controllare sia PIU PICCOLO, di tutti i suoi successivi.

Mi servono quindi due indici, i e j, per riferirmi ad una coppia di elementi. Mi serve che j sia sempre minore di i, perché mi interessano solo gli elementi SUCCESSIVI a quello considerato.

Mi interessa che gli elementi puntati da **i** siano **più** distanti di quelli puntati da **j, rispetto alla location «loc» indicata**

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null
- **@ ensures** (\exists Site s; getSites().contains(s); s.getType().equals(type)) && \result!=null &&

(\forall Site s; ; \result.contains(s) <==> getSites().contains(s) && s.getType().equals(type)) &&

(\forall int i; i>=0 && i<\result.size();

(\forall int j; j>=0 && j<i; !\result.get(j).equals(\result.get(i)) && \result.get(j).getLocation().getDistance(loc)<=\result.get(i).getLocation().getDistance(loc)))

- ***@ signals*** (NotFoundException e) !(\exists Site s; getSites().contains(s); s.getType().equals(type))

TdE: 28-08-2023 Esercizio 1 – soluzione a

- **@ requires** type!=null && loc!=null
- **@ ensures** (\exists Site s; getSites().contains(s); s.getType().equals(type)) && \result!=null &&

(\forall Site s; ; \result.contains(s) <==> getSites().contains(s) && s.getType().equals(type)) &&

(\forall int i; i>=0 && i<\result.size();

(\forall int j; j>=0 && j<i; !\result.get(j).equals(\result.get(i)) && \result.get(j).getLocation().getDistance(loc)<=\result.get(i).getLocation().getDistance(loc)))

- ***@ signals*** (NotFoundException e) !(\exists Site s; getSites().contains(s); s.getType().equals(type))

TdE: 28-08-2023 Esercizio 1 – quesito b

- Si specifichi in JML il metodo getNearestSites

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

TdE: 28-08-2023 Esercizio 1 – soluzione b

@requires

TdE: 28-08-2023 Esercizio 1 – soluzione b

@requires k>0 && loc!=null

Per prima cosa, siamo certi che loc deve essere diverso da null, e k deve essere maggiore di 0, quindi partiamo con le precondizioni

TdE: 28-08-2023 Esercizio 1 – soluzione b

@requires k>0 && loc!=null

Per prima cosa, siamo certi che loc deve essere diverso da null, e k deve essere maggiore di 0, quindi partiamo con le precondizioni

Dal momento che non vengono specificate exceptions, sappiamo anche che non dovremo usare alcun signal

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

TdE: 28-08-2023 Esercizio 1 – soluzione b

@requires k>0 && loc!=null

@ensures

TdE: 28-08-2023 Esercizio 1 – soluzione b

@requires k>0 && loc!=null

@ensures \result!=null && \result.size() == (k<getSites().size() ? k : getSites().size()) &&

Il risultato non può essere null, e questa è la prima condizione. Inoltre: se k è maggiore del numero complessivo di Site, li restituisce tutti. Quindi possiamo anche mettere una condizione sulla dimensione di \result

TdE: 28-08-2023 Esercizio 1 – soluzione b

@requires k>0 && loc!=null

@ensures \result!=null && \result.size() == (k<getSites().size() ? k : getSites().size()) &&

TdE: 28-08-2023 Esercizio 1 – soluzione b

@requires k>0 && loc!=null

@ensures \result!=null && \result.size() == (k<getSites().size() ? k : getSites().size()) &&

(\forall Site s; \result.keySet().contains(s);

getSites().contains(s) && \result.get(s)==s.getLocation().getDistance(loc)) &&

Altra cosa di cui siamo certi è che, essendo una Map<Site,Double>, ogni chiave della mappa, ovvero ogni Site, deve essere uno dei Site della CityMap e, il valore corrispondente alla chiave, Deve essere la sua distanza dalla locazione (loc) indicata.

Nota: notare l'uso di \result.get(s) per accedere al valore della Map data la chiave

TdE: 28-08-2023 Esercizio 1 – soluzione b

L’ultimo controllo che resta da fare dice:
Ritorna i k punti di interesse piu vicini alla locazione indicata.

TdE: 28-08-2023 Esercizio 1 – soluzione b

L’ultimo controllo che resta da fare dice:
Ritorna i k punti di interesse piu vicini alla locazione indicata.

Quindi, **se un site S** della CityMap**,** ha meno di **K** Site s1 che sono più vicini rispetto alla locazione loc, allora **s** rientra nei K siti più vicini a loc.

In altre parole, data una locazione L, se un sito A ha meno di N siti più vicini ad L rispetto a lui, allora A è **nei primi N siti** vicini ad L

TdE: 28-08-2023 Esercizio 1 – soluzione b

L'ultimo controllo che resta da fare dice:
Ritorna i k punti di interesse piu vicini alla locazione indicata.

Quindi, **se un site S** della CityMap**,** ha meno di **K** Site s1 che sono più vicini rispetto alla locazione loc, allora **s** rientra nei K siti più vicini a loc.

In altre parole, data una locazione L, se un sito A ha meno di N siti più vicini ad L rispetto a lui, allora A è **nei primi N siti** vicini ad L

In parole un po' più simili alla specifica JML, per ogni sito S1 che appartiene all'insieme dei siti, se ci sono meno di K siti S1 appartenenti all'insieme dei siti che sono più vicini a loc, allora S fa parte dell'output, poiché è nei primi K siti più vicini

TdE: 28-08-2023 Esercizio 1 – soluzione b

@ requires k>0 && loc!=null

@ ensures \result!=null && \result.size() == (k<getSites().size() ? k : getSites().size()) &&

(\forall Site s; \result.keySet().contains(s);

getSites().contains(s) && \result.get(s)==s.getLocation().getDistance(loc)) &&

TdE: 28-08-2023 Esercizio 1 – soluzione b

@ requires k>0 && loc!=null

@ ensures \result!=null && \result.size() == (k<getSites().size() ? k : getSites().size()) &&

(\forall Site s; \result.keySet().contains(s);

getSites().contains(s) && \result.get(s)==s.getLocation().getDistance(loc)) &&

(\forall Site s; getSites().contains(s);

(\num Site s1; getSites().contains(s1) && !s1.equals(s);

s1.getLocation().getDistance(loc) < s.getLocation().getDistance(loc)) < k ==> \result.keySet().contains(s))

TdE: 28-08-2023 Esercizio 1 – soluzione b

@ requires k>0 && loc!=null

@ ensures \result!=null && \result.size() == (k<getSites().size() ? k : getSites().size()) &&

(\forall Site s; \result.keySet().contains(s);

getSites().contains(s) && \result.get(s)==s.getLocation().getDistance(loc)) &&

(\forall Site s; getSites().contains(s);

(\num Site s1; getSites().contains(s1) && !s1.equals(s);

s1.getLocation().getDistance(loc) < s.getLocation().getDistance(loc)) < k ==> \result.keySet().contains(s))

TdE: 28-08-2023 Esercizio 1 – quesito c

- Per proseguire con il quesito c, occorre conoscere gli Abstract Data Type.

Abstract Data Type:

Idea: JML consente di descrivere i metodi. Fino ad ora, abbiamo utilizzato i metodi puri per descrivere i metodi modificatori….
Ma chi descrive i metodi puri?

Abstract Data Type:

Idea: JML consente di descrivere i metodi. Fino ad ora, abbiamo utilizzato i metodi puri per descrivere i metodi modificatori….
Ma chi descrive i metodi puri?

I tipi di dato astratto si basano sul trovare un parametro, o comunque qualcosa che consenta di descrivere i metodi puri

Abstract Data Type:

Il **Representation Invariant** esprime tutto ciò che **è sempre vero** degli attributi di una classe. Va definito per tutti quegli attributi che vengono poi utilizzati per descrivere i metodi puri.

Abstract Data Type:

Il **Representation Invariant** esprime tutto ciò che **è sempre vero** degli attributi di una classe. Va definito per tutti quegli attributi che vengono poi utilizzati per descrivere i metodi puri.

La **Funzione di Astrazione,** invece, mi consente di descrivere l'output dei metodi puri, in funzione di quelli che sono gli attributi utilizzati nel representation invariant.

Rep Invariant vs Public Invariant

- Il **rep invariant** è una condizione che deve essere vera in ogni momento in cui si usa un'istanza della classe- In altre parole è una proprietà sempre valida in ogni momento, indipendentemente dalle operazioni eseguite sulla classe. Riguarda principalmente **lo stato interno** del Sistema.
- Il **public invariant** invece è una condizione che deve essere vera ogni qual volta l'oggetto della classe **esegue** delle operazioni pubbliche. In alter parole, il public invariant riflette le condizioni che devono essere valide quando si accede all'oggetto dalla prospettiva degli utenti ESTERNI alla classe.
  Questo può includere non solo il rep invariant, ma anche proprietà oggettive, utili all'utente.

Invariants

Representation Invariant

Private Invariant

Public Invariant

Representation Invariant

- A ***representation invariant*** describes a property that must always hold for the internal representation of an object, regardless of external visibility. It applies internally to the class and ensures data consistency.

- **Visibility**: private to the class.
- **Purpose**: ensures that the internal structure is valid and consistent.
- **Usage**: validates implementation details, such as the correctness of private fields.

Representation Invariant - Example

Let’s consider a bank account. You have:

-**Balance ->** Amount of Money on your bank account

-**Threshold ->** Max allowed money on your bank account

// Representation invariant: balance must not exceed threshold

// @ private invariant
balance <= threshold

Private Invariant

A *private invariant* is similar to a *representation invariant*, but it concerns **only the private members** of the class. It is **tightly tied to implementation details** and is **not visible** outside the class.

- **Visibility**: private to the class.
- **Purpose**: checks conditions specific to private members.
- **Usage**: used to define conditions that should not be exposed or considered by subclasses or external objects.

Private Invariant - Example

Let’s consider a bank account. You have:

- **Balance ->** Amount of Money on your bank account
- **Threshold ->** Max allowed money on your bank account

// Private invariant: threshold must always be non-negative

//@ private invariant

threshold >= 0;

Public Invariant

A ***public invariant*** specifies a **property that must remain true** for an object **and is visible to external users**.

Clients of the class **can rely on this property** being valid **at all times**.

- **Visibility**: public, so it is visible to external users.
- **Purpose**: defines public properties that describe the visible behavior of the object.
- **Usage**: used to provide guarantees to the class's clients.

Public Invariant - Example

Let’s consider a bank account. You have:

- **Balance ->** Amount of Money on your bank account
- **Threshold ->** Max allowed money on your bank account

// Public invariant: balance cannot be negative
//@ public invariant balance >= 0

TdE: 28-08-2023 Esercizio 1 – quesito c

- Proviamo allora a risolvere il quesito c.

Si consideri un'implementazione che fa uso di una mappa per memorizzare i punti di interesse organizzati per tipo, come mostrato di seguito.

```
public class CityMap {
   private Map<Type, Set<Site>> sites;
... }
```

Per tale implementazione di forniscano l'invariante di rappresentazione e la funzione di astrazione.

TdE: 28-08-2023 Esercizio 1 – quesito c

- Proviamo allora a risolvere il quesito c.

Si consideri un'implementazione che fa uso di una mappa per memorizzare i punti di interesse organizzati per tipo, come mostrato di seguito.

```
public class CityMap {
   private Map<Type, Set<Site>> sites;
... }
```

Per tale implementazione di forniscano l'invariante di rappresentazione e la funzione di astrazione.

TdE: 28-08-2023 Esercizio 1 – soluzione c

- Quali sono quelle proprietà che **«sono vere»** quando parlo della mappa **sites**?

TdE: 28-08-2023 Esercizio 1 – soluzione c

- **L'invariante di rappresentazione (RI)** deve specificare che la mappa non sia nulla,

RI:

- //@ private invariant
  sites!=null

TdE: 28-08-2023 Esercizio 1 – soluzione c

- **L'invariante di rappresentazione (RI)** deve specificare che la mappa non sia nulla, non contenga chiavi nulle,

.

RI:

- //@ private invariant
  sites!=null && !sites.containsKey(null) &&

TdE: 28-08-2023 Esercizio 1 – soluzione c

- **L'invariante di rappresentazione (RI)** deve specificare che la mappa non sia nulla, non contenga chiavi nulle, non contenga valori nulli o vuoti.

RI:

- //@ private invariant
  sites!=null && !sites.containsKey(null) &&
  (\forall Type t; sites.containsKey(t); sites.get(t)!=null &&
  !sites.get(t).isEmpty() &&

TdE: 28-08-2023 Esercizio 1 – soluzione c

- **L'invariante di rappresentazione (RI)** deve specificare che la mappa non sia nulla, non contenga chiavi nulle, non contenga valori nulli o vuoti. Inoltre ogni punto di interesse deve essere non nullo

RI:

- //@ private invariant
  sites!=null && !sites.containsKey(null) &&
  (\forall Type t; sites.containsKey(t); sites.get(t)!=null &&
  !sites.get(t).isEmpty() && (\forall Site s; sites.get(t).contains(s);
  s!=null

TdE: 28-08-2023 Esercizio 1 – soluzione c

- **L'invariante di rappresentazione (RI)** deve specificare che la mappa non sia nulla, non contenga chiavi nulle, non contenga valori nulli o vuoti. Inoltre ogni punto di interesse deve essere non nullo e memorizzato nell'insieme corrispondente al proprio tipo.

RI:

- //@ private invariant
  sites!=null && !sites.containsKey(null) &&
  (\forall Type t; sites.containsKey(t); sites.get(t)!=null &&
  !sites.get(t).isEmpty() && (\forall Site s; sites.get(t).contains(s);
  s!=null && s.getType().equals(t)))

TdE: 28-08-2023 Esercizio 1 – soluzione c

- Che cosa deve definire la funzione di astrazione ?

TdE: 28-08-2023 Esercizio 1 – soluzione c

- Che cosa deve definire la funzione di astrazione ?

Deve definire quei metodi che ho usato per **specificare le mie procedure.**

Quale metodo abbiamo usato?

TdE: 28-08-2023 Esercizio 1 – soluzione c

- Che cosa deve definire la funzione di astrazione ?

Deve definire quei metodi che ho usato per **specificare le mie procedure.**

Quale metodo abbiamo usato? getSites()

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

TdE: 28-08-2023 Esercizio 1 – soluzione c

- La funzione di astrazione (AF) si limita a definire **getSites** in funzione di **sites**, ovvero in funzione dell'implementazione esplicitata dalla traccia, in quanto tutti gli altri metodi sono specificati a partire da getSites().

- //@ private invariant

  (\forall Site s; ; getSites().contains(s) <==>

  sites.containsKey(s.getType()) && sites.get(s.getType()).contains(s))

TdE 19-02-2019

Si consideri il tipo di dato astratto Connect4 che modella una partita al noto gioco “Forza 4”. Lo scopo del gioco `e allineare quattro pedine dello stesso colore nella scacchiera di gioco sia in verticale, orizzontale o obliquo. A questo scopo due giocatori (uno dotato di 21 pedine gialle, l’altro di 21 pedine rosse) si alternano facendo scivolare la pedina nei binari prestabiliti dalla scacchiera.
Si supponga una dimensione della “griglia” come in figura (7 colonne per 6 righe).
Una pedina viene depositata invocando il metodo put ed indicando la colonna in cui si vuole depositare la pedina (nell’intervallo 1-7) e l’identificativo del giocatore.
I due giocatori sono convenzionalmente identificati dai valori interi 1 e 2. Il metodo get ritorna 0 se la cella indicata `e vuota, oppure 1 o 2 a seconda della pedina presente nella posizione indicata (del giocatore 1 o del giocatore 2). La cella in basso a sinistra ha posizione 1,1.

TdE 19-02-2019

```
public class Connect4 {
    // Crea una partita vuota.
    public Connect4() ;

    // Deposita la pedina del giocatore player (1 o 2) nella colonna indicata.
    // Restituisce l'eccezione FullColumnException se la colonna è piena.
    // Restituisce l'eccezione GameOverException se c'è già un vincitore.
    public void put(int column, int player) throws FullColumnException, GameOverException;

    // Restituisce 0 se la cella è libera, 1 o 2 se è occupata dalla pedina
    // del giocatore corrispondente. La posizione indicata deve appartenere
    // all'intervallo ammesso (1-7 x 1-6). La cella in basso a sinistra (vedi figura)
    // ha posizione 1,1.
    public /*@ pure @*/ int get(int column, int row) ;

    // Restituisce true sse c'è un vincitore (4 pedine dello stesso giocatore allineate).
    public /*@ pure @*/ boolean winner() ;

    // Svuota la griglia dando inizio a una nuova partita.
    public void clear() ;
}
```

TdE 19-02-2019 : Quesito 2

- Si specifichi l’invariante pubblico per il tipo Connect4

TdE 19-02-2019 : Quesito 2 - soluzione

Ragioniamo sulle regole: cosa è sempre vero?

1. Una cella è sempre vuota (0), con una pedina del giocatore 1 (1), oppure con una pedina del giocatore 2 (2).

TdE 19-02-2019 : Quesito 2 - soluzione

Ragioniamo sulle regole: cosa è sempre vero?

1. Una cella è sempre vuota (0), con una pedina del giocatore 1 (1), oppure con una pedina del giocatore 2 (2).

```
//@public invariant

\forall(int c; c>=1 && c<=7; \forall(int r; r>=1 && r<=6; ( get(c,r)==0 || get(c,r)==1 || get(c,r)==2 ) &&
```

TdE 19-02-2019 : Quesito 2 - soluzione

Ragioniamo sulle regole: cosa è sempre vero?

2. Se una casella, in posizione (3,3), è 1, allora tutte le caselle nella stessa colonna devono essere non vuote (altrimenti la pedina non si terrebbe, in posizione (3,3) e cadrebbe

TdE 19-02-2019 : Quesito 2 - soluzione

Ragioniamo sulle regole: cosa è sempre vero?

2. Se una casella, in posizione (3,3), è 1, allora tutte le caselle nella stessa colonna devono essere non vuote (altrimenti la pedina non si terrebbe, in posizione (3,3) e cadrebbe

```
&& get(c,r)!=0 => \forall(int r1; r1>=1 && r1<=r; get(c,r1)!=0)))) &&
```

TdE 19-02-2019 : Quesito 2 - soluzione

Ragioniamo sulle regole: cosa è sempre vero?

3. Inoltre, il numero di pedine in campo differisce sempre al più di 1, in quanto si gioca un turno a testa.

TdE 19-02-2019 : Quesito 2 - soluzione

Ragioniamo sulle regole: cosa è sempre vero?

3. Inoltre, il numero di pedine in campo differisce sempre al più di 1, in quanto si gioca un turno a testa.

-1<=(\sumof(int c; c>=1 && c<=7; \numof(int r; r>=1 && r<=6; get(c,r)==1)) - \sumof(int c; c>=1 && c<=7; \numof(int r; r>=1 && r<=6; get(c,r)==2)))<= 1

TdE 19-02-2019 : Quesito 2 - soluzione

Ragioniamo sulle regole: cosa è sempre vero?

Soluzione

```
//@ public invariant \forall(int c; c>=1 && c<=7; \forall(int r; r>=1 && r<=6;
//@       ( get(c,r)==0 || get(c,r)==1 || get(c,r)==2 ) &&
//@       ( get(c,r)!=0 => \forall(int r1; r1>=1 && r1<=r; get(c,r1)!=0)))) &&
//@       -1<=( \sumof(int c; c>=1 && c<=7; \numof(int r; r>=1 && r<=6; get(c,r)==1))
//@           - \sumof(int c; c>=1 && c<=7; \numof(int r; r>=1 && r<=6; get(c,r)==2)) )<= 1
```

TdE 16/01/2018

- Data la seguente classe:

```
public class Bunch<T> {
// OVERVIEW: Una collezione di oggetti di tipo T.
// Gli elementi sono disposti in ordine qualunque e sono  di molteplicità qualunque.
// Il valore null non  fa parte della collezione
// Usa equals per confrontare gli oggetti

//@ensures (*costruisce il bunch vuoto*)
public Bunch();

// Aggiunge x a this
// Lancia eccezione se x e' null
public void ins(T x) throws NullPointerException;

// Elimina una comparsa di x
// Restituisce il numero di elementi rimanenti uguali a x.
// Lancia eccezione se x e' null
public int del(T x)  throws NullPointerException;

// Restituisce il numero di elementi uguali a x
public /*@ pure @*/ int num(T x)
```

TdE 16/01/2018 – quesito a

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
\\@Public Invariant
\forall(T t;;num(t) > = 0);
```

TdE 16/01/2018 – quesito a - soluzione

- Scrivere la specifica JML del metodo del(T x) e un invariante pubblico per la classe.

```
//@ensures
        x!=null &&
        (\forall T t;!t.equals(x); num(t) == \old(num(t)) &&
                \old(num(x)==0) ? num(x) ==0 && \result==0
                        num(x) ==\old(num(x)-1) && \result==num(x);
//@signals(NullPointerException e) x == null;
public int del(T x) throws NullPointerException
```

TdE 16/01/2018 – quesito b

- Completare, con AF e RI (esprimibili con opportuni inviarianti) e con il codice del metodo del(T x), la seguente implementazione della classe Bunch. Il rep è costituito da una lista di coppie, ciascuna contenente un elemento, se presente, e la sua molteplicità. I dettagli possono essere ricavati dall'esame del codice.

TdE 16/01/2018 – quesito b

```
public class Bunch<T> {
//rep:
     private List<Pair> elem;
     private static class Pair {
          // OVERVIEW: Tipo immutabile rappresentante una coppia (T,int)
          T t;
          int i;
          Pair(T y, int j) {
               t = y;
               i = j;
          }

          boolean contains(T x) {
               return (x.equals(t));
          }
     }
```

```
public Bunch(){
     elem = new ArrayList<Pair>();
}
private int find(T x) {
     for (int j=0; j<elem.size(); j++;) {
          Pair p = elem.get(j);
          if (p.contains(x)) return j;
     }
     return -1;
}
public /*@ pure @*/ int num(T x) {
     int j = find(x);
     if (j==-1) return 0;
     return elem.get(j).i;
}
public void ins(T x) throws NullPointerException {
     if (x==null) throw new NullPointerException();
     i = find(x);
     if (i>-1) elem.set(i,new Pair(x,1+elem.get(i).i));
     else elem.add(new Pair(x,1));
}
public int del(T x) throws NullPointerException {
     if (x==null) throw new NullPointerException();
     int j = find(x);
     if (j==-1) return 0;

     }
 }
```

TdE 16/01/2018 – quesito b - soluzione

- Per prima cosa, implementiamo il metodo:

TdE 16/01/2018 – quesito b - soluzione

- Per prima cosa, implementiamo il metodo:

```
public int del(T x) throws NullPointerException {
  if (x==null) throw new NullPointerException();
  int j = find(x);
  if (j==-1) return 0;
  int temp=elem.get(j).i-1; // tolgo 1, al numero di comparse di x
  if (temp==0) {  // se il valore scende a 0, lo rimuovo
    elem.remove(j);
  } else { // altrimenti aggiorno
    elem.set(j,new Pair(x,temp));
  }
  return temp;
}
```

TdE 16/01/2018 – quesito b - soluzione

- Ora passiamo al RI. Il representation invariant mi dice un qualcosa che è «sempre vero» negli attributi della classe. In questo caso, la lista è diversa da null e il valore di occorrenze, se l'elemento è inserito nella lista, è sempre maggiore di 0

TdE 16/01/2018 – quesito b - soluzione

- Ora passiamo al RI. Il representation invariant mi dice un qualcosa che è «sempre vero» negli attributi della classe. In questo caso, la lista è diversa da null e il valore di occorrenze, se l'elemento è inserito nella lista, è sempre maggiore di 0

Soluzione

```
private invariant elem != null &&
(\forall int j; 0<=j && j<elem.size();
 elem.get(j).i>0);
```

TdE 16/01/2018 – quesito b - soluzione

- AF, invece, è quella funzione che mi permette di esprimere i metodi pure. In questo caso, dobbiamo descrivere l'output di num()

Rivediamo la funzione

```
public Bunch(){
     elem = new ArrayList<Pair>();
}
private int find(T x) {
     for (int j=0; j<elem.size(); j++;) {
          Pair p = elem.get(j);
          if (p.contains(x)) return j;
     }
     return -1;
}
public /*@ pure @*/ int num(T x) {
     int j = find(x);
     if (j==-1) return 0;
     return elem.get(j).i;
}
public void ins(T x) throws NullPointerException {
     if (x==null) throw new NullPointerException();
     i = find(x);
     if (i>-1) elem.set(i,new Pair(x,1+elem.get(i).i));
     else elem.add(new Pair(x,1));
}
public int del(T x) throws NullPointerException {
     if (x==null) throw new NullPointerException();
     int j = find(x);
     if (j==-1) return 0;

     }
 }
```

TdE 16/01/2018 – quesito b - soluzione

- AF, invece, è quella funzione che mi permette di esprimere i metodi pure. In questo caso, dobbiamo descrivere l'output di num()

TdE 16/01/2018 – quesito b - soluzione

- AF, invece, è quella funzione che mi permette di esprimere i metodi pure. In questo caso, dobbiamo descrivere l'output di num()

Soluzione

```
private invariant
(\forall int j; 0<=j && j<elem.size();
    num(elem.get(j).t)==elem.get(j).i) &&
```

Per ogni elemento t, num è pari al valore di get.i e…

TdE 16/01/2018 – quesito b - soluzione

- AF, invece, è quella funzione che mi permette di esprimere i metodi pure. In questo caso, dobbiamo descrivere l'output di num()

Soluzione

```
private invariant
(\forall int j; 0<=j && j<elem.size();
    num(elem.get(j).t)==elem.get(j).i) &&
(\forall T x;;
    !(\exists int j; 0<=j && j<elem.size(); x.equals(elem.get(j).t)
    ==> num(x)==0;
```

Per ogni x, se non esiste una j tale che elem.get(j).t sia uguale ad X, allora X non è stato trovato, quindi il risultato di num deve essere 0

TdE 16/01/2018 – quesito b - soluzione

- AF, invece, è quella funzione che mi permette di esprimere i metodi pure. In questo caso, dobbiamo descrivere l'output di num()

Soluzione

```
private invariant
(\forall int j; 0<=j && j<elem.size();
    num(elem.get(j).t)==elem.get(j).i) &&
(\forall T x;;
    !(\exists int j; 0<=j && j<elem.size(); x.equals(elem.get(j).t)
    ==> num(x)==0;
```